UD4 – TÉCNICAS DE PROGRAMACIÓN SEGURA

Contenido

1 Conceptos clave de seguridad 1
2 Criptografía 4
3 Hash 4
4 Autenticación 7
1.1 Autenticación basada Usuario y Contraseña 7
5 Autorización basada en Claims 15
6 Uso de JSON Web Tokens 18
7 Encriptación con clave simétrica 19
8 Encriptación con clave simétrica 22
9 Buenas Prácticas de Programación Segura 25

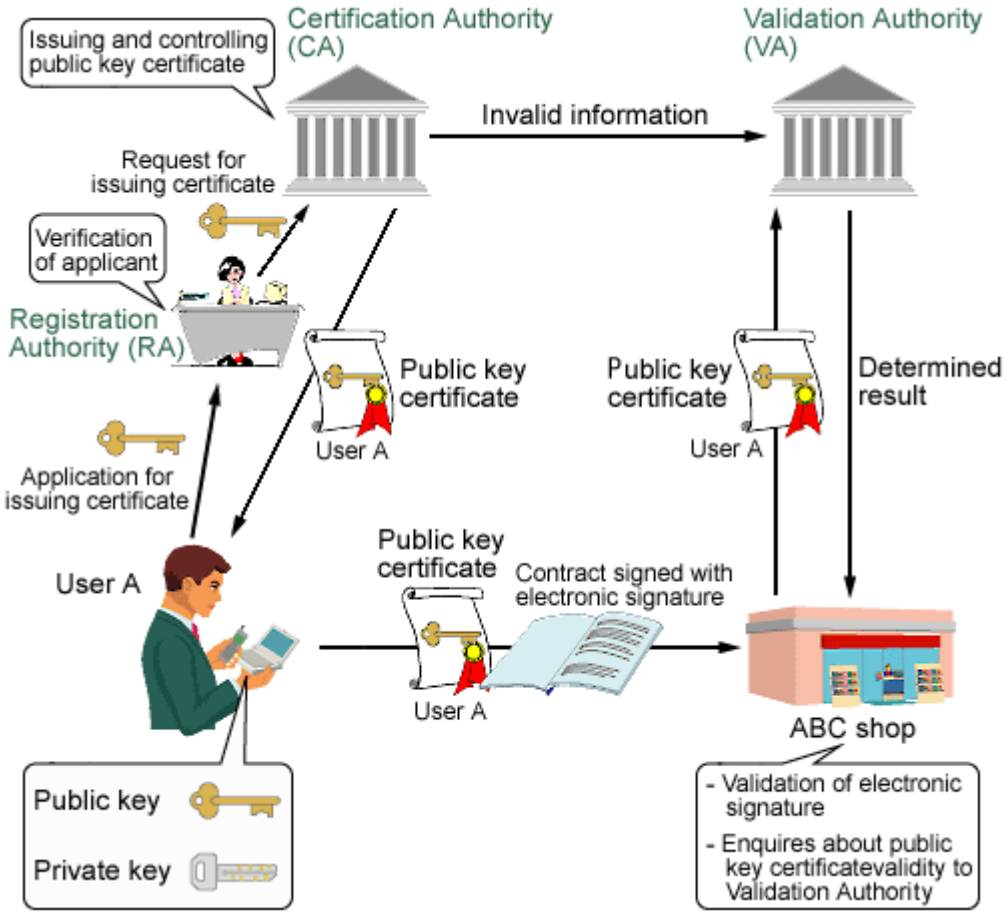
1 Conceptos clave de seguridad

Empezaremos estudiando una serie de conceptos clave referidos a la seguridad:

- **Integridad**. Se trata de poder garantizar la totalidad y la exactitud de la información que se gestiona.
- **Confidencialidad**. Implica que debe protegerse la información de forma que sólo sea conocida por las personas autorizadas y se garantice que terceras personas no pueden acceder a ellas.
- **Disponibilidad**. La disponibilidad implica que debe protegerse la información de forma tal que se pueda disponer de ella para su gestión en el tiempo y la forma requeridos por el usuario.
- **Identificación**. Proceso mediante el cual el sujeto suministra información diciendo quién es.
- **Autenticación**. Proceso por el que se verifica que alguien es quién dice ser. Por ejemplo, mediante usuario y contraseña, con tarjeta inteligente, huella dactilar, reconocimiento facial etc.
- **Autorización**. Proceso por el cual, una vez identificado un sujeto, se determina si tiene acceso o no a un determinado recurso.
- **No repudio**. Se trata de evitar que el autor de un documento pueda negar que él ha generado dicho documento.

- **Criptografía**. La criptografía, escritura secreta, del griego, kryptos y graphein, se encarga de estudiar cómo ocultar el significado de un mensaje mediante el cifrado de éste para que únicamente sea accesible al destinatario legítimo del mismo.
- **Cifrado**. Proceso por el que el texto legible o texto en claro es transformado de manera que no sea comprensible a toda persona que no tenga la clave para descifrarlo. Existen **algoritmos** diversos para transformar texto en claro en texto cifrado, en general haciendo uso de una **clave** de cifrado.
- **Criptografía simétrica**. Conjunto de técnicas de cifrado basadas en el uso de la misma clave de cifrado y descifrado compartida por emisor y receptor.
- **Criptografía asimétrica**. Conjunto de técnicas de cifrado basadas en el uso de un par de claves; una pública y otra privada. El emisor cifra los mensajes haciendo uso de la clave pública difundida por el receptor, entonces el mensaje sólo podrá ser descifrado por el poseedor de la clave privada, que estará custodiada por el receptor.
- **Algoritmos de una sola vía. Hash**. Se trata de obtener un resumen o hash a partir de un documento o mensaje original. No es imposible que dos mensajes den como resultado el mismo Hash, pero es altísimamente improbable que podamos manipular un documento cambiando su información y el resultado sea un documento que produzca el mismo hash. Tampoco es posible obtener el documento original a partir del hash. Se utiliza para garantizar la integridad de la información y en los procesos de firma digital.
- **Firma digital**. Una firma digital está compuesta por una serie de datos asociados a un mensaje con el objetivo de asegurar la identidad del emisor y la integridad del mensaje. El proceso es el siguiente:
    - El emisor genera un mensaje.
    - El emisor genera un hash de dicho mensaje y lo cifra con su clave privada. Por lo tanto, cualquiera que conozca la clave pública podrá descifrar el hash. El resultado es lo que se conoce como **firma digital**.
    - El emisor envía mensaje y la firma digital de dicho mensaje.
    - El receptor separa el mensaje y la firma digital que recibe.
    - El receptor calcula el hash del mensaje con el mismo algoritmo.
    - El receptor descifra la firma digital recibida con la clave pública del emisor.
    - Si ambos hashs coinciden queda garantizado que el mensaje fue enviado el propietario de la clave pública utilizada y que el mensaje no ha sido modificado en el transcurso de la comunicación.
- **Certificado digital**. Se trata de un documento que certifica que una entidad determinada tiene una clave pública determinada. El certificado tiene que ser capaz de enlazar una clave pública junto al nombre del titular del certificado.
- **Autoridad de certificación. CA**. Entidad que se encarga de emitir y gestionar certificados digitales. Se trata de entidades de confianza. La forma que tienen las CA de hacer válido un certificado es firmarlo digitalmente.

- **X.509**. Formato estándar para certificados digitales. Tiene los siguientes campos:
    - Versión del formato del certificado
    - Número de serie, id en la CA
    - Algoritmo de firma y parámetros
    - Emisor del certificado
    - Fechas de inicio y fin de la validez del certificado
    - Nombre del propietario del certificado
    - Identificador del algoritmo
    - Firma digital del certificado de la AC



2 Criptografía

.NET proporciona implementaciones de numerosos algoritmos criptográficos estándar. Estos algoritmos son fáciles de usar y tienen las propiedades predeterminadas más seguras. Además, el modelo objetos de criptografía de .NET es muy extensible.

Microsoft en su documentación hace la siguiente recomendación para la elección de algoritmos en función de la aplicación:

- Privacidad de los datos:
  - Aes
- Integridad de los datos:
  - HMACSHA256
  - HMACSHA512
- Firma digital:
  - ECDsa
  - RSA
- Intercambio de claves:
  - ECDiffieHellman
  - RSA
- Generación de números aleatorios:
  - RNGCryptoServiceProvider
- Generar una clave a partir de una contraseña:
  - Rfc2898DeriveBytes

3 Hash

Un valor hash es un valor numérico de longitud fija que solo identifica datos. Los valores hash representan grandes cantidades de datos como valores numéricos mucho menores, por lo que se usan con firmas digitales. Podemos firmar un valor hash de forma más eficaz que una cantidad de datos mayor. Los valores hash también son útiles para comprobar la integridad de datos enviados a través de canales no seguros. El valor hash de los datos recibidos puede compararse con el valor hash enviado para determinar si se alteraron los datos.

Vamos a ver cómo generar y comprobar códigos hash mediante las clases en el espacio de nombres System.Security.Cryptography.

Las clases hash administradas pueden aplicar un valor hash a una matriz de bytes o a un objeto de secuencia administrado. En el ejemplo, utilizaremos SHA256 para crear un valor hash para una cadena y después para comprobar el hash obtenido.

Debido a problemas de colisión con SHA1, se recomienda utilizar SHA256 o superior.

```
import java.nio.charset.StandardCharsets;

import java.security.MessageDigest;

import java.security.NoSuchAlgorithmException;

import java.util.Base64;


public class HashUtil {

    public static String getHash(String msg) {

        try {

            // Convertimos el mensaje en un array de bytes UTF-16 (equivalente a Unicode)

            byte[] msgBytes = msg.getBytes(StandardCharsets.UTF_16LE);

            // Creamos una instancia de SHA-256

            MessageDigest digest = MessageDigest.getInstance("SHA-256");

            // Obtenemos el hash

            byte[] hashValue = digest.digest(msgBytes);

            // Convertimos el hash a Base64 y lo devolvemos

            return Base64.getEncoder().encodeToString(hashValue);

        } catch (NoSuchAlgorithmException e) {

            // Esto no debería ocurrir ya que SHA-256 siempre existe

            throw new RuntimeException(e);

        }

    }

    // Método de prueba

    public static void main(String[] args) {

        String mensaje = "Hola Mundo";

        String hash = getHash(mensaje);

        System.out.println("Hash: " + hash);

    }

}
```

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.Base64;
public class HashUtil {
    public static String getHash(String msg) {
        try {
            byte[] msgBytes = msg.getBytes(StandardCharsets.UTF_16LE); // Unicode C# ? UTF-16LE
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hashValue = digest.digest(msgBytes);
            return Base64.getEncoder().encodeToString(hashValue);
        } catch (NoSuchAlgorithmException e) {
            throw new RuntimeException(e);
    }}
    public static boolean checkHash(String msg, String hash) {
        try {
            // Convertimos el mensaje en bytes
            byte[] msgBytes = msg.getBytes(StandardCharsets.UTF_16LE);
            // Convertimos el hash Base64 en bytes
            byte[] hashBytes = Base64.getDecoder().decode(hash);
            // Calculamos el hash del mensaje
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hashValue = digest.digest(msgBytes);
            // Comprobamos si los arrays son iguales
            if (hashValue.length != hashBytes.length) {
                return false;
            }
            for (int i = 0; i < hashBytes.length; i++) {
                if (hashValue[i] != hashBytes[i]) {
                    return false;
           }}
            return true;
        } catch (NoSuchAlgorithmException e) {
            throw new RuntimeException(e);
    }}
    // Método de prueba
    public static void main(String[] args) {
        String mensaje = "Hola Mundo";
        String hash = getHash(mensaje);
        System.out.println("Hash: " + hash);
        System.out.println("Check: " + checkHash(mensaje, hash)); // debería ser true
    }}
```

```
public class Program {
    public static void main(String[] args) {
        String msg = "Hola Mundo!";
        String hash = HashUtil.getHash(msg);  // Llamamos al método que genera el hash
        if (HashUtil.checkHash(msg, hash)) { // Comprobamos el hash
            System.out.println("El hash coincide");
        } else {
            System.out.println("ERROR! El hash no coincide");
        }
    }
}
```

4 Autenticación

Podremos elegir entre diferentes tipos de autenticación cuando creemos una aplicación con .NET:

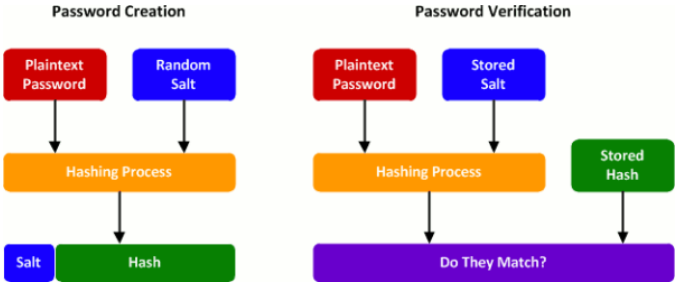
- La autenticación de **Windows** es especialmente conveniente para aplicaciones de intranet donde todos los usuarios están registrados en el Active Directory y se trabaja dentro del Firewall de la compañía. Cuando la aplicación requiera autenticación sólo tiene que usar la información del Active Directory e iniciar sesión automáticamente. No hay necesidad para hacer que el usuario inicie sesión de manera separada en la aplicación cuando ya ha iniciado sesión en su ordenador.
- La manera más común de autenticación de usuarios en una aplicación es a través de un formulario (**Form**) de inicio de sesión, típicamente utilizando nombre de usuario (o email) y contraseña. Las contraseñas se validan contra una fuente de datos, como por ejemplo una base de datos SQL. Una vez autenticado correctamente el usuario accede a la aplicación. En una aplicación de escritorio, se guardará la información del usuario en algún objeto en memoria. En una aplicación web se podría guardar cierta información en las cookies para mantener la información de la sesión y no repetir el inicio de sesión en cada acción.
- **OpenAuth** significa que existe una aplicación externa que es responsable de la autenticación de los usuarios. Estamos acostumbrados a iniciar sesión en un sitio web a través de las credenciales de Google, Microsoft, Facebook o Yahoo entre otros. En estos casos corresponde al operador respectivo autenticar al usuario en lugar de a la propia aplicación.

1.1 Autenticación basada Usuario y Contraseña

La manera más común de autenticar a un usuario en una aplicación es a través de un formulario y pidiendo usuario y contraseña. El mecanismo es simple, se guarda la información de las credenciales en una base de datos, el usuario introduce la

información se valida contra la información de la base de datos y en caso de ser correcta el usuario inicia se sesión, el siguiente paso será dar autorización al usuario a las distintas funcionalidades de acuerdo a los permisos otorgados a dicho usuario.

La información de inicio de sesión es muy sensible y se producen ataques frecuentes a los sitios web tratando de obtener usuarios y contraseñas, así que es necesario proteger esta información. La técnica que se considera actualmente adecuada para proteger las contraseñas es conocida como "***Salted Password Hashing***".



El flujo de proceso es básicamente el siguiente:

Creación de las credenciales

1. El usuario crea una cuenta introduciendo un nombre de usuario
2. El usuario introduce una contraseña
3. El sistema crea una secuencia aleatoria llamada sal (salt) cuya misión es esconder la contraseña y hacer dificultar los ataques de diccionario y los ataques basados en tablas de contraseñas.
4. Se concatena la contraseña a la sal, habitualmente *salt + password*.
5. Se obtiene un valor hash para la concatenación anterior y se guarda en la base de datos el nombre de usuario, la sal y el hash

Verificación de las credenciales

1. El usuario introduce usuario y contraseña
2. La información se envía al servidor
3. Se recupera sal y hash correspondiente a dicho usuario
4. Se concatena la contraseña a la sal y se obtiene el valor hash
5. Se compara el hash obtenido con el hash guardado en la base de datos
6. Si coinciden los hashs la autenticación es correcta, en caso contrario se deniega el acceso.

Buenas prácticas

1. Nunca se reutiliza la sal; se ha de generar una nueva secuencia aleatoria cada vez que se establece una nueva contraseña.
2. La información de inicio de sesión se envía al servidor y es en el servidor donde se concatena a la sal y se calcula el hash para la validación. Nunca se valida en el cliente y se envía el hash calculado al servidor, si se hiciera así el hash estaríamos en la misma situación que guardar la contraseña en claro en la base

de datos y si un atacante obtuviera los hashs podría iniciar sesión fácilmente enviado el hash al servidor.

3. El servidor no debe dar información adicional si la credencial es incorrecta, es decir no se debe dar información de si el usuario existe o no.
4. La sal tiene que ser suficientemente larga. Una regla simple es utilizar una sal de la misma longitud que la salida de la función hash. Por ejemplo, la salida del algoritmo SHA256 es 256 bits, es decir, 32 bytes, entonces crearemos una sal de 32 bytes de longitud.
5. Para generar la sal debe usarse un generador CSPRNG (Cryptographically Secure Pseudo-Random Number Generator), por ejemplo en C# tenemos disponible System.Security.Cryptography.RNGCryptoServiceProvider.
6. Deben utilizarse algoritmos lentos para la obtención del valor hash, como por ejemplo **PBKDF2** implementado por la clase Rfc2898DeriveBytes, de esta forma se dificultan los ataques de fuerza bruta.

7. Se puede reforzar la seguridad guardando el hash cifrado con clave secreta de forma que se dificulte la utilización de la información en caso de una brecha de seguridad exponga la información de las credenciales ya que se necesitaría obtener también la clave secreta para poder usar los hashs.

Veámoslo con un ejemplo:

```
// Representa un registro con usuario y hash de la contraseña
public class Credential {
    private String user;
    private String passwordHash;
    public Credential(String user, String passwordHash) {
        this.user = user;
        this.passwordHash = passwordHash;
    }
    // Getters y Setters
    public String getUser() {
        return user;
    }
    public void setUser(String user) {
        this.user = user;
    }
    public String getPasswordHash() {
        return passwordHash;
    }
    public void setPasswordHash(String passwordHash) {
        this.passwordHash = passwordHash;
    }
}
// Representa un usuario conectado (solo lectura)
public class LoginSesion {
    private final String user;

    public LoginSesion(String user) {
        this.user = user;
    }
    // Solo getter, sin setter para mantenerlo inmutable
    public String getUser() {
        return user;
    }
}
```

```
import java.security.NoSuchAlgorithmException;

import java.security.SecureRandom;

import java.security.spec.InvalidKeySpecException;

import javax.crypto.SecretKeyFactory;

import javax.crypto.spec.PBEKeySpec;

public class PasswordUtil {

    // Generar un salt de manera aleatoria

    public static byte[] generateSalt() {

        byte[] salt = new byte[32]; // 32 bytes

        SecureRandom random = new SecureRandom();

        random.nextBytes(salt); // llenamos el array con bytes aleatorios

        return salt;

    }

    // Obtener el hash de la contraseña usando PBKDF2

    public static byte[] getHash(String password, byte[] salt) {

        final int ITERATIONS = 1000;

        final int KEY_LENGTH = 256; // 32 bytes * 8 bits = 256 bits

        try {

            PBEKeySpec spec = new PBEKeySpec(password.toCharArray(), salt, ITERATIONS,
KEY_LENGTH);

            SecretKeyFactory skf = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");

            return skf.generateSecret(spec).getEncoded();

        } catch (NoSuchAlgorithmException | InvalidKeySpecException e) {

            throw new RuntimeException(e);

        }

    }

    // Método de prueba

    public static void main(String[] args) {

        byte[] salt = generateSalt();

        byte[] hash = getHash("MiContraseñaSegura", salt);

        System.out.println("Salt: " + java.util.Base64.getEncoder().encodeToString(salt));

        System.out.println("Hash: " + java.util.Base64.getEncoder().encodeToString(hash));

    }

}
```

```
import java.util.Base64; import java.util.Scanner;
public class AccountUtil {
    private static final Scanner scanner = new Scanner(System.in);
    public static Credential createAccount() {
        System.out.println("Creación de una nueva cuenta");
        // Crear usuario
        String user = "";
        while (user.isEmpty()) {
            System.out.println("Introduce nombre de usuario:");
            user = scanner.nextLine().trim();
        }
        // Crear contraseña y confirmarla
        String password = "";
        boolean match = false;
        while (!match) {
            // Introducir contraseña
            while (password.isEmpty()) {
                System.out.println("Introduce contraseña:");
                password = scanner.nextLine();       }
            // Confirmar contraseña
            String confirm = "";
            while (confirm.isEmpty()) {
                System.out.println("Confirma la contraseña:");
                confirm = scanner.nextLine();}
            match = password.equals(confirm);
            if (!match) {
                System.out.println("Las contraseñas no coinciden. Vuelve a intentarlo.");
                password = ""; // Reiniciamos para volver a pedir   }}
        // Generar salt y hash
        byte[] salt = PasswordUtil.generateSalt();
        byte[] hash = PasswordUtil.getHash(password, salt);
        // Guardamos salt + hash en un solo array de 64 bytes
        byte[] saveHash = new byte[64];
        System.arraycopy(salt, 0, saveHash, 0, 32);
        System.arraycopy(hash, 0, saveHash, 32, 32);
        // Convertimos a Base64 para almacenar
        String passwordHash = Base64.getEncoder().encodeToString(saveHash);
        return new Credential(user, passwordHash);
    }
```

```
import java.util.Base64; import java.util.HashMap; import java.util.Map; import
java.util.Scanner;

public class Auth {

    private static final Scanner scanner = new Scanner(System.in);

    private static final Map<String, Credential> credentials = new HashMap<>();

    public static boolean login(LoginSesion[] userSession) {

        // Inicializamos la sesión vacía

        userSession[0] = null;

        System.out.println("Inicio de sesión");

        // Introducir usuario y contraseña

        System.out.println("Introduce nombre de usuario:");

        String user = scanner.nextLine().trim();

        System.out.println("Introduce la contraseña:");

        String password = scanner.nextLine(); // no ocultamos la contraseña aquí

        if (credentials.containsKey(user)) {

            // Obtener credencial

            Credential credential = credentials.get(user);

            // Extraer salt y hash del string Base64

            byte[] savedHash = Base64.getDecoder().decode(credential.getPasswordHash());

            byte[] salt = new byte[32];

            byte[] hash = new byte[32];

            System.arraycopy(savedHash, 0, salt, 0, 32);

            System.arraycopy(savedHash, 32, hash, 0, 32);

            // Calcular hash de la contraseña ingresada

            byte[] checkHash = PasswordUtil.getHash(password, salt);

            // Comparar byte a byte

            for (int i = 0; i < hash.length; i++) {

                if (hash[i] != checkHash[i]) {

                    return false; // hash no coincide  }         }

            // Hash coincide: devolvemos la sesión

            userSession[0] = new LoginSesion(user);

            return true;

        } else {

            // Usuario no existe: simulamos validación para evitar retardo de timing

            byte[] salt = PasswordUtil.generateSalt();

            PasswordUtil.getHash(password, salt);

            return false;  }  }
```

```
import java.util.HashMap;
import java.util.Map;

public class Program {
    // Representa la "tabla de usuarios" de la aplicación
    private static final Map<String, Credential> credentials = new HashMap<>();

    public static void main(String[] args) {
        // Creación de una nueva cuenta
        Credential credential = AccountUtil.createAccount();
        // Guardamos la credencial para validar los inicios de sesión
        credentials.put(credential.getUser(), credential);
        System.out.println();
        // Inicio de sesión
        LoginSesion[] login = new LoginSesion[1]; // simula 'out' de C#
        int attempts = 3;
        boolean success = false;

        while (attempts > 0) {
            if (Auth.login(login)) {
                success = true;
                break;
            } else {
                System.out.println("Credenciales no válidas\n");
                attempts--;
            }
        }
        if (success) {
            System.out.println("Usuario " + login[0].getUser() + " conectado");
        } else {
            System.out.println("Superado número de intentos. No conectado");
        }
    }
}
```

5 Autorización basada en Claims

Un "*claim*" en el contexto de autenticación y autorización es una afirmación (o una declaración) acerca de una entidad, típicamente un usuario. Un *claim* da una información más refinada que la pertenencia o no a un rol, por ejemplo:

- El nombre de usuario de Mary es mary.smith
- Mary es del departamento IT
- Mary está en el grupo *Administrator*
- El email de mary es msmith@outlook.com
- El centro de trabajo de Mary está en Redmond
- Mary tiene permisos para editar usuarios del dominio
- Mary tiene permisos para autorizar órdenes de compra

El tipo claim es básicamente un par clave-valor:

- Username: mary.smith
- Department: IT
- Rol: Administrator
- Email: msmith@outlook.com
- Center: Redmon
- DomainUser: Edition
- SalesOrder: Supervisor

Vemos que el concepto de rol que hemos visto en el punto anterior se ajusta perfectamente a un tipo claim, pero éstos son mucho más expresivos que únicamente indicar el listado de roles a los que pertenece; IT, SalesSupervisor, etc. De esta forma será mucho más fácil refinar los permisos de un sistema o aplicación que la simple definición de roles.

Los claims deberán provenir de una entidad de confianza distinta de la que está siendo descrita. Es decir, no basta con que Mary diga que es administradora, si por ejemplo la intranet de la compañía está basada en Windows, el Active Directory, podría guardar esta información sobre los claims de Mary. La idea es que exista una entidad de confianza de la que obtengamos la información de autorización de un usuario. Esta entidad de confianza que informa sobre una información es conocida como Issuer. En la pareja clave-valor de un claim, la clave es conocida como tipo y el valor es conocido como valor. Por ejemplo:

- Tipo: rol
- Valor: Administrador
- Issuer: Active Directory

La confianza es muy importante en un sistema como este; habitualmente confiamos en el Active Directory de nuestra red, pero para una entidad externa necesitaremos algo más para confiar, como por ejemplo un certificado digital del issuer.

Una de las ventajas del modelo de seguridad en claims es que puede utilizar distintos sistemas de autenticación que podrán informar sobre distintos grupos de claims: grupos, roles, permisos, etc.

Un claim está representado en .NET (desde el framework 4.5) por una clase llamada Claim presente en el namespace System.Security.Claims

Veamos un ejemplo simple de creación de un objeto ClaimsIdentity

```
import java.util.ArrayList;

import java.util.List;

// Representa un Claim (tipo + valor)

class Claim {

   private final String type;

   private final String value;

   public Claim(String type, String value) {

      this.type = type;

      this.value = value;

   }

   public String getType() { return type; }

   public String getValue() { return value; }

}

// Representa una identidad con una lista de claims

class ClaimsIdentity {

   private final List<Claim> claims;

   public ClaimsIdentity(List<Claim> claims) {

      this.claims = claims;

   }

   public List<Claim> getClaims() { return claims; }

}

// Representa el principal (usuario) con una identidad

class ClaimsPrincipal {

   private final ClaimsIdentity identity;

   public ClaimsPrincipal(ClaimsIdentity identity) {

      this.identity = identity;

   }

   public ClaimsIdentity getIdentity() { return identity; }

}
```

```
// Clase para simular el hilo actual (como Thread.CurrentPrincipal)
class ThreadPrincipal {
   private static ClaimsPrincipal currentPrincipal;
   public static void setPrincipal(ClaimsPrincipal principal) {
      currentPrincipal = principal;
   }
   public static ClaimsPrincipal getPrincipal() {
      return currentPrincipal;
   }
}
// Ejemplo de uso
public class ClaimsExample {
   public static void main(String[] args) {
      String user = "user123";
      List<Claim> claims = new ArrayList<>();
      // Claims predefinidos
      claims.add(new Claim("NameIdentifier", user));
      claims.add(new Claim("Name", "Joe"));
      claims.add(new Claim("Surname", "Brown"));
      claims.add(new Claim("Role", "Operator"));
      claims.add(new Claim("Country", "US"));
      claims.add(new Claim("NameIdentifier", user));
      // Claim personalizado
      claims.add(new Claim("Time", "Select"));
      ClaimsIdentity identity = new ClaimsIdentity(claims);
      ClaimsPrincipal principal = new ClaimsPrincipal(identity);
      // Asignamos al "hilo actual"
         ThreadPrincipal.setPrincipal(principal);


      // Ejemplo: mostrar claims
      System.out.println("Claims del usuario:");
      for (Claim c : ThreadPrincipal.getPrincipal().getIdentity().getClaims()) {
         System.out.println(c.getType() + " = " + c.getValue());
      }
   }
}
```

El siguiente vídeo explica cómo funciona el mecanismo de autenticación basada en Claims: How Claims Based Authentication works

Las clases de .NET proporcionan funcionalidad y compatibilidad completa con la autenticación basada en claims.

6 Uso de JSON Web Tokens

Un token es una unidad de datos que no tienen significado por sí mismo, pero que en combinación con un sistema de "tokenización" se convierte en un actor vital en la seguridad de las aplicaciones. La autenticación basada en tokens funciona asegurando que cada petición a un servidor está acompañada de un token firmado cuya autenticidad será verificada por el servidor antes de responder a la petición.

JSON Web Token (JWT) es un estándar abierto (RFC 7519) que define un método compacto autocontenido para la transmisión segura de información entre partes. Utiliza formato JSON para la construcción de los tokens. JWT se ha hecho popular gracias a su tamaño compacto que permite una fácil comunicación vía petición de cadenas de caracteres, atributos de cabecera o en el propio cuerpo de una petición post.

El uso de tokens ofrece varias ventajas frente a los métodos tradicionales tales como el uso de cookies:

- El token es autocontenido, por lo que contiene toda la información necesaria para la autenticación. Lo que facilita la escalabilidad al liberar al servidor de guardar la información de la sesión.
- Los tokens pueden ser generados en cualquier sitio. La generación de un token está desacoplada de su verificación permitiendo la opción de firmar tokens en un servidor diferente o incluso a través de una compañía diferente tales como una OAuth.
- Permiten configurar el control de acceso a un nivel más bajo especificando los roles y permisos asociados al usuario.

Los tokens constan de 3 partes: Header, Payload y Signature. Cabecera y carga útil se codifican en Base64 y se concatenan con un punto. Después se firma obteniendo una cadena de caracteres con la forma *header.payload.signature*.

Veamos el funcionamiento de los jwt:

- En la autenticación, cuando el usuario inicia sesión correctamente con sus credenciales, se devolverá un jwt.
- Cuando el usuario desee acceder a una ruta o recurso protegido, se debe enviar el jwt, normalmente en el encabezado de la llamada mediante el esquema *Bearer*. El contenido del encabezado debe ser similar al siguiente:

```
Authorization: Bearer <token>
```

- Las rutas protegidas del servidor comprobarán si hay un JWT válido en el encabezado y, si está presente, el usuario podrá acceder a los recursos protegidos. El JWT contendrá la información necesaria (claims) para autorizar o no el acceso a un recurso en concreto.
- Para poder verificar la autenticidad del token se necesitará una clave para poder validar la firma (signatura) del token. En la cabecera del token se especifica el algoritmo utilizado en la firma. Se puede utilizar cifrado simétrico (SHA) o asimétrico (RSA). Si utilizamos el primero tendremos el problema de la distribución de la clave. Utilizando RSA además de asegurarnos de que el token no ha sido manipulado estaremos asegurando el origen del mismo.

Este mecanismo permite un sistema de autorización sin estado.

Buenas prácticas en el uso de tokens:

- La clave de firma de los tokens debe tratarse como cualquier otra credencial y debe estar disponible sólo para aquellos servicios para los que es absolutamente necesaria.
- No se debe añadir información sensible al payload.
- Los tokens deben tener una caducidad.
- No se deben enviar tokens en conexiones no seguras, como http sin TLS.
- En las aplicaciones cliente servidor, además de las verificaciones llevadas a cabo en el cliente, el servidor deberá implementar un sistema de verificación para asegurar el origen del token.

Puedes encontrar más información en JSON Web Tokens - jwt.io

7 Encriptación con clave simétrica

Las clases de criptografía simétrica administrada se usan con una clase de secuencia especial llamada CryptoStream que cifra los datos leídos en la secuencia. La clase *CryptoStream* se inicializa con una clase de secuencia administrada, una clase que implementa la interfaz ICryptoTransform (creada a partir de una clase que implementa un algoritmo criptográfico) y una enumeración CryptoStreamMode que describe el tipo de acceso permitido a *CryptoStream*. La clase *CryptoStream* puede inicializarse usando cualquier clase que derive de la clase Stream, incluidas FileStream, MemoryStream y NetworkStream. Mediante estas clases, podemos realizar el cifrado simétrico en diversos objetos de secuencia.

En el ejemplo siguiente se muestra cómo crear una nueva instancia de la clase RijndaelManaged, que implementa el algoritmo de cifrado **Rijndael** (o **AES**), y usarla para realizar el cifrado en una clase *CryptoStream*. En este ejemplo, vamos a cifrar cierta información en un fichero y después leeremos y descifraremos el contenido del fichero. Para descifrar es necesario utilizar la misma clave y el mismo vector de inicialización utilizados en el cifrado:

```
import javax.crypto.Cipher;

import javax.crypto.CipherInputStream;

import javax.crypto.CipherOutputStream;

import javax.crypto.spec.IvParameterSpec;

import javax.crypto.spec.SecretKeySpec;

import java.io.*;

public class MainCrypto {

    // Clave de cifrado (16 bytes = 128 bits)

    private static final byte[] key = {

            0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x10, 0x11, 0x12,
0x13, 0x14, 0x15, 0x16

    };

    // Vector de inicialización (IV)

    private static final byte[] iv = {

            0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x10, 0x11, 0x12,
0x13, 0x14, 0x15, 0x16

    };

    // Ruta del fichero

    private static final String path = "archivo_cifrado.dat";

    public static void main(String[] args) {

        cifrado();

        descifrado();

    }
```

```
// Método que llevará a cabo el cifrado

    private static void cifrado() {

        try {

            // Creamos stream hacia el fichero

            FileOutputStream fileOut = new FileOutputStream(path);

            // Creamos el cifrador AES/CBC/PKCS5Padding

            Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");

            SecretKeySpec secretKey = new SecretKeySpec(key, "AES");

            IvParameterSpec ivSpec = new IvParameterSpec(iv);

            cipher.init(Cipher.ENCRYPT_MODE, secretKey, ivSpec);

            // Creamos un flujo cifrado

            CipherOutputStream cipherOut = new CipherOutputStream(fileOut, cipher);

            BufferedWriter writer = new BufferedWriter(new OutputStreamWriter(cipherOut,
"UTF-8"));

            // Escribimos texto

            writer.write("Este es mi texto en claro");

            writer.newLine();

            // Cerramos flujos

            writer.close();

            cipherOut.close();

            fileOut.close();

            System.out.println("Fichero cifrado");

        } catch (Exception e) {

            System.out.println("Error en el cifrado");

            e.printStackTrace();

        }

    }
```

```
// Método de descifrado
    private static void descifrado() {
        try {
            // Stream de lectura del fichero
            FileInputStream fileIn = new FileInputStream(path);
            // Configuramos el descifrado
            Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
            SecretKeySpec secretKey = new SecretKeySpec(key, "AES");
            IvParameterSpec ivSpec = new IvParameterSpec(iv);
            cipher.init(Cipher.DECRYPT_MODE, secretKey, ivSpec);
            // Flujo de descifrado
            CipherInputStream cipherIn = new CipherInputStream(fileIn, cipher);
            BufferedReader reader = new BufferedReader(new InputStreamReader(cipherIn, "UTF-
8"));
            System.out.println("Texto en claro contenido en el fichero:");
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
            // Cerramos flujos
            reader.close();
            cipherIn.close();
            fileIn.close();
        } catch (Exception e) {
            System.out.println("Lectura fichero errónea");
            e.printStackTrace();
        }
    }
}
```

8 Encriptación con clave simétrica

La clase RSACryptoServiceProvider realiza el cifrado y el descifrado asimétrico mediante la implementación del algoritmo RSA.

Veamos cómo crear claves asimétricas con esta clase:

```
import java.nio.charset.StandardCharsets;

import java.security.*;

import java.security.spec.*;

import java.util.Base64;

import javax.crypto.Cipher;

public class RSAUtil {

    // Clase para almacenar par de claves

    public static class RSAKeys {

        public final String privateKey;

        public final String publicKey;

        public RSAKeys(String privateKey, String publicKey) {

            this.privateKey = privateKey;

            this.publicKey = publicKey;

        }

    }

    // Crear un par de claves RSA

    public static RSAKeys createRSAKeys(int size) {

        try {

            KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");

            keyGen.initialize(size);

            KeyPair pair = keyGen.generateKeyPair();

            String privateKey =
Base64.getEncoder().encodeToString(pair.getPrivate().getEncoded());

            String publicKey =
Base64.getEncoder().encodeToString(pair.getPublic().getEncoded());

            return new RSAKeys(privateKey, publicKey);

        } catch (NoSuchAlgorithmException e) {

            throw new RuntimeException(e);

        }

    }
```

Una vez creadas las claves, los datos encriptados con una clave sólo podrán ser desencriptados con la otra clave. De esta forma podemos asegurar comunicaciones seguras encriptando con la clave pública de manera que sólo el destinatario correcto podrá descifrar el mensaje. También podremos usar las claves para firmar documentos. Se puede firmar un documento con clave privada, de manera que podemos verificar la firma con la clave pública y comprobar que el origen del documento es el poseedor de la clave privada.

Veamos cómo cifrar con clave pública:

```
// Cifrar texto con clave pública

    public static String encrypt(String text, String base64PublicKey) {

        try {

            byte[] keyBytes = Base64.getDecoder().decode(base64PublicKey);

            X509EncodedKeySpec spec = new X509EncodedKeySpec(keyBytes);

            KeyFactory kf = KeyFactory.getInstance("RSA");

            PublicKey publicKey = kf.generatePublic(spec);

            Cipher cipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");

            cipher.init(Cipher.ENCRYPT_MODE, publicKey);

            byte[] bytes = text.getBytes(StandardCharsets.UTF_16LE); // equivalente a
Encoding.Unicode

            byte[] cipherBytes = cipher.doFinal(bytes);

            return Base64.getEncoder().encodeToString(cipherBytes);

        } catch (Exception e) {

            throw new RuntimeException(e);

        }}
```

Y cómo descifrar con clave privada:

```
// Descifrar texto con clave privada

    public static String decrypt(String cipherText, String base64PrivateKey) {

        try {

            byte[] keyBytes = Base64.getDecoder().decode(base64PrivateKey);

            PKCS8EncodedKeySpec spec = new PKCS8EncodedKeySpec(keyBytes);

            KeyFactory kf = KeyFactory.getInstance("RSA");

            PrivateKey privateKey = kf.generatePrivate(spec);

            Cipher cipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");

            cipher.init(Cipher.DECRYPT_MODE, privateKey);

            byte[] cipherBytes = Base64.getDecoder().decode(cipherText);

            byte[] decrypted = cipher.doFinal(cipherBytes);

            return new String(decrypted, StandardCharsets.UTF_16LE);

        } catch (Exception e) {

            throw new RuntimeException(e);

        }}
```

```
// Ejemplo de uso
    public static void main(String[] args) {
        RSAKeys keys = createRSAKeys(2048);
        System.out.println("Clave pública: " + keys.publicKey);
        System.out.println("Clave privada: " + keys.privateKey);
        String mensaje = "Hola Mundo con RSA";
        String cifrado = encrypt(mensaje, keys.publicKey);
        System.out.println("Cifrado: " + cifrado);
        String descifrado = decrypt(cifrado, keys.privateKey);
        System.out.println("Descifrado: " + descifrado);
    }
}
```

9 Buenas Prácticas de Programación Segura

A continuación, se expone una pequeña lista de indicaciones relativas a la seguridad en programación.

- La primera buena práctica es mantenerse informado, manejar información actualizada sobre los protocolos y algoritmos que se consideran seguros e inseguros.
- Limpiar datos y código.
- No guardar contraseñas en ficheros o bases de datos sin cifrar.
- No transmitir información sensible por canales no seguros.
- No fiarse de los recibidos desde los clientes (url, cookies, formularios) ya que pueden ser manipulados. Realizar siempre comprobaciones siempre en el lado del servidor, aunque parezcan redundantes.
- Utilizar algoritmos criptográficos estándar, no debemos crear los nuestros propios.
- No invocar a programas no confiables desde nuestros programas.
- No invocar a un Shell o línea de comandos desde nuestros programas.
- No se deben mostrar las contraseñas por pantalla.
- No enviar contraseñas vía email u otro medio.
- No distribuir información confidencial por correo electrónico.
- Seguir los protocolos establecidos en la organización.
- Cuidado con el uso de Eval o la generación dinámica de código o de sentencias SQL. En caso de ser necesario se debe analizar el código resultante antes de ejecutarlos para evitar ataques de inyección.

Recuerda, mantente siempre informado de las recomendaciones de seguridad que afecten a la tarea que vas a realizar.